

Git y GitHub: El Ciclo de Vida del Desarrollo Profesional

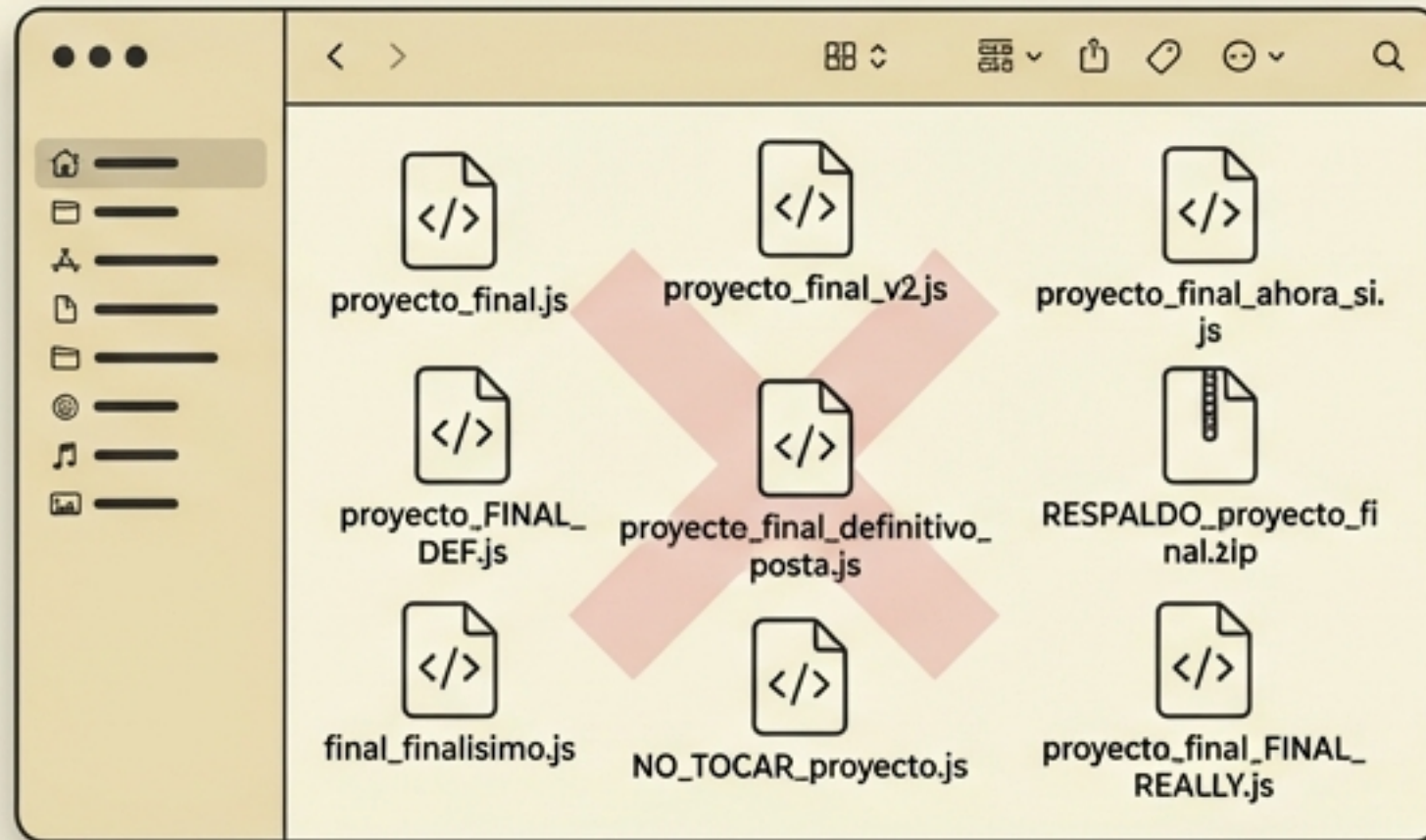
De la teoría local al flujo de trabajo colaborativo en la nube.



Guía integral para desarrolladores de software.

¿Por qué necesitamos una Máquina del Tiempo?

THE CHAOS



THE ORDER



El Problema

Sin control, trabajar en equipo es imposible. Se sobrescribe el trabajo, se pierden versiones estables y no hay rastro de quién hizo qué.

La Solución

Un Sistema de Control de Versiones (SCV) funciona como un 'cuaderno con máquina del tiempo' para tu código.

La Realidad

El 99.9% de las empresas tecnológicas usan Git. Es un requisito crítico para manejar proyectos de millones de dólares.

Distinguiendo la Herramienta de la Plataforma



Git (La Cámara)

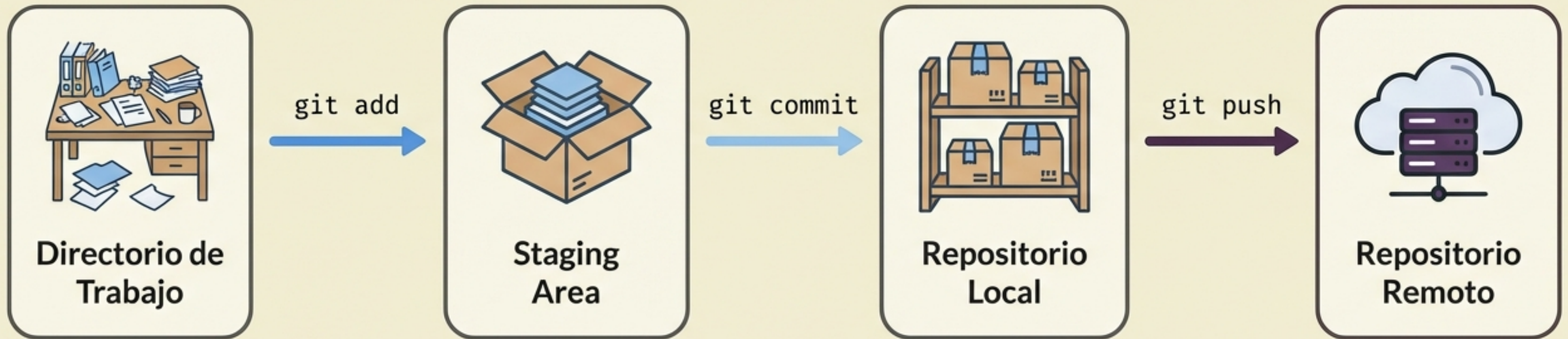
- Software instalado localmente.
- Toma “fotografías” instantáneas de tu código.
- Funciona sin internet.
- Creado por Linus Torvalds.



GitHub (El Álbum)

- Servicio web para guardar tus fotos.
- Es el “Google Photos” de tu código.
- Permite compartir y colaborar.
- Permite compartir y colaborar.
- Alternativas: GitLab, Bitbucket.

La Estación de Empaque: Los 4 Estados



Leyenda

- **Untracked:** Archivo nuevo, desconocido para Git.
- **Modified:** Archivo con cambios sin guardar.
- **Staged:** Listo para ser confirmado (en la caja).
- **Committed:** Guardado permanentemente en el historial.

Paso 1: Configuración e Inicialización

```
$  
$ git config --global user.name 'Tu Nombre'  
$ git config --global user.email 'tu@email.com'  
$ git init  
$
```

La Firma (Config)

Define quién eres. Estás firmando tu nombre en cada "fotografía" que tomarás en el futuro.

El Inicio (Init)

Crea una carpeta oculta .git/. Es el equivalente a sacar una cámara nueva de su caja.

Nota: Solo usa init si empiezas de cero. Si descargas un proyecto existente, usarás clone.

Paso 2: La Brújula del Proyecto (Git Status)



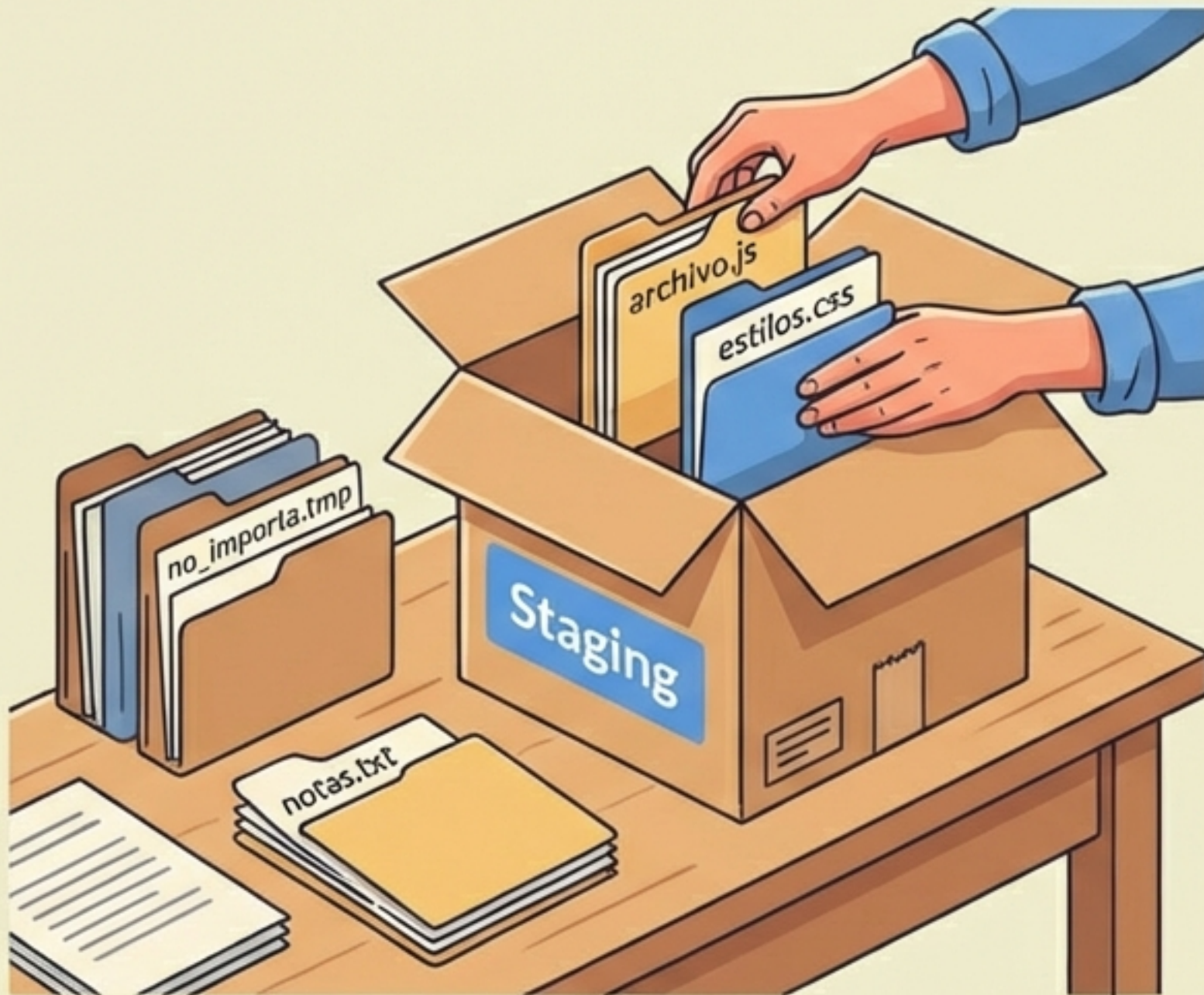
¿Qué hace?: Le pregunta a la cámara: "¿Qué fotos tengo pendientes por tomar?"

Úsalo todo el tiempo: Es tu mejor amigo para no perderte.

● Rojo (Untracked/Modified): Cambios en tu mesa de trabajo, no listos.

● Verde (Staged): Cambios en la caja de empaque, listos para el commit.

Paso 3: Empacando la Caja (Git Add)



Mueve archivos del Directorio de Trabajo al Área de Preparación (Staging).

No estás obligado a enviar todo lo que modificaste; tú eliges qué entra en la "caja".

```
git add archivo.js  # Agrega uno específico
git add .           # Agrega TODO (Cuidado
                    con archivos innecesarios)
```

Paso 4: Tomando la Fotografía (Git Commit)



Guarda permanentemente los cambios del Staging en el historial local.

Cada commit es un punto de retorno en el tiempo.

Una vez hecho el commit, la 'caja' (Staging) se vacía y el contenido queda guardado en tu álbum local.

```
git commit -m 'Descripción del cambio'
```

Escribiendo la Historia: El Mensaje de Commit

El mensaje debe completar la frase: “Si aplico este commit, va a...”

Malos Mensajes

arreglos

asdfgh

cambios

no sé qué hice

Buenos Mensajes

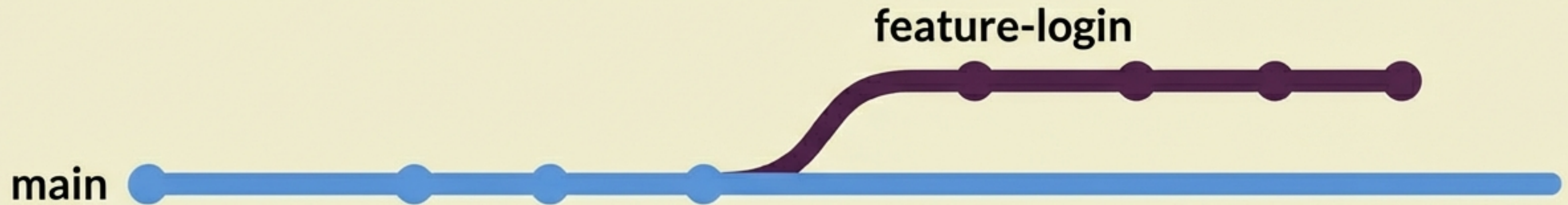
Agrega función de login

Corrige bug en cálculo de impuestos

Actualiza dependencias

Un buen historial permite a tu equipo entender la evolución del proyecto sin leer todo el código.

Universos Paralelos: Ramas (Git Branch)



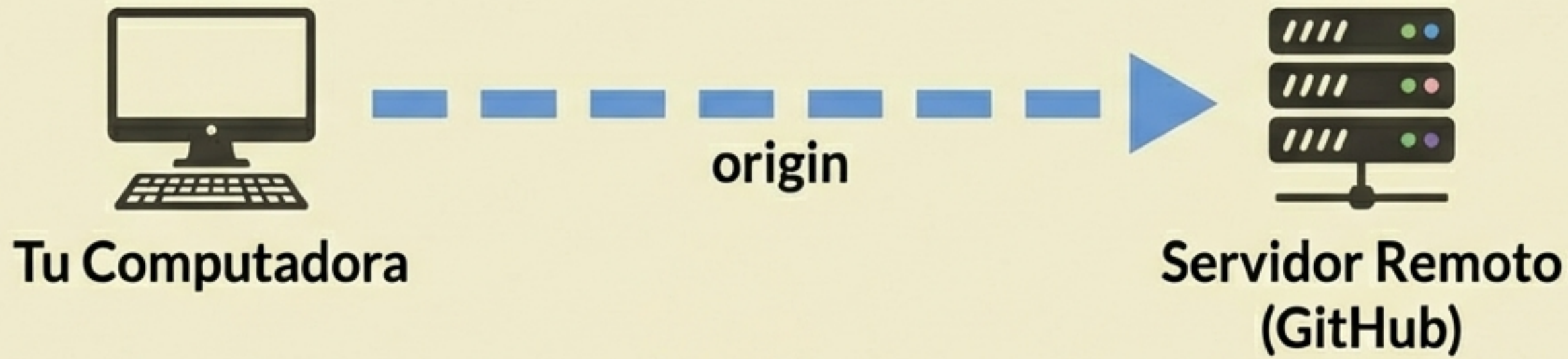
Permite probar nuevas funciones sin romper el código original.

Main: La versión oficial y estable.

Feature Branches: Donde ocurre la magia y el desarrollo diario.

```
git branch nombre-rama      # Crear universo
git checkout nombre-rama    # Viajar al universo
git checkout -b nueva-rama   # Crear y viajar en un paso
```

Conectando con la Nube (Git Remote & Push)



- **git remote add origin URL:** Agrega la dirección de envío a tu paquete. "Origin" es solo el **nombre convencional del servidor**.
- **git push:** Envía tus commits locales al servidor remoto.

```
git push -u origin main
```

← El switch **-u** establece la conexión por defecto para el futuro.

Descargando Historia (Clone vs. Pull)

Git Clone



- **Analogía:** Descargas el álbum de fotos completo por primera vez.
- **Uso:** Al unirte a un proyecto nuevo.

```
git clone  
https://github.com/usuario/repo.git
```

Git Pull



- **Analogía:** Tu compañero subió nuevas fotos y tú actualizas tu copia local.
- **Uso:** Diariamente, antes de empezar a trabajar.

```
git pull origin main
```

Revisando y Restaurando el Pasado



```
git log --oneline
```

Muestra el historial de commits de forma compacta.



```
git diff
```

Muestra exactamente qué líneas de código han cambiado antes de hacer commit.



```
git restore archivo.js
```

Deshace cambios en un archivo (¡Peligroso! Borra el trabajo no guardado).

Trabajo en Equipo: El Pull Request (PR)



Definición: Una solicitud para que tu código sea revisado y fusionado en la rama principal.

Analogía: El Periódico



1. **Tu Rama:** Tu escritorio donde escribes el artículo.



2. **Pull Request:** Entregas el borrador al editor.



3. **Code Review:** El editor sugiere correcciones.

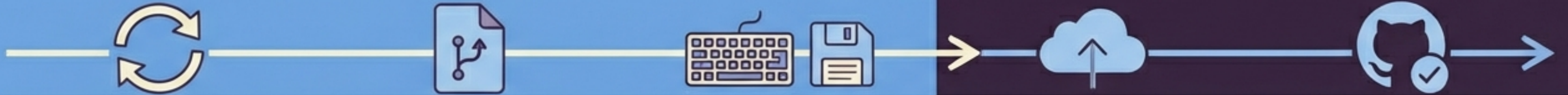


4. **Merge:** El artículo se publica en la edición impresa (Main).

El Ciclo de Vida de una Funcionalidad (Feature)

Fase Local

Fase Remota



Sincronizar

Crear Rama

Trabajar y Guardar

Subir Rama

Solicitar Revisión

```
git checkout  
main  
+ git pull
```

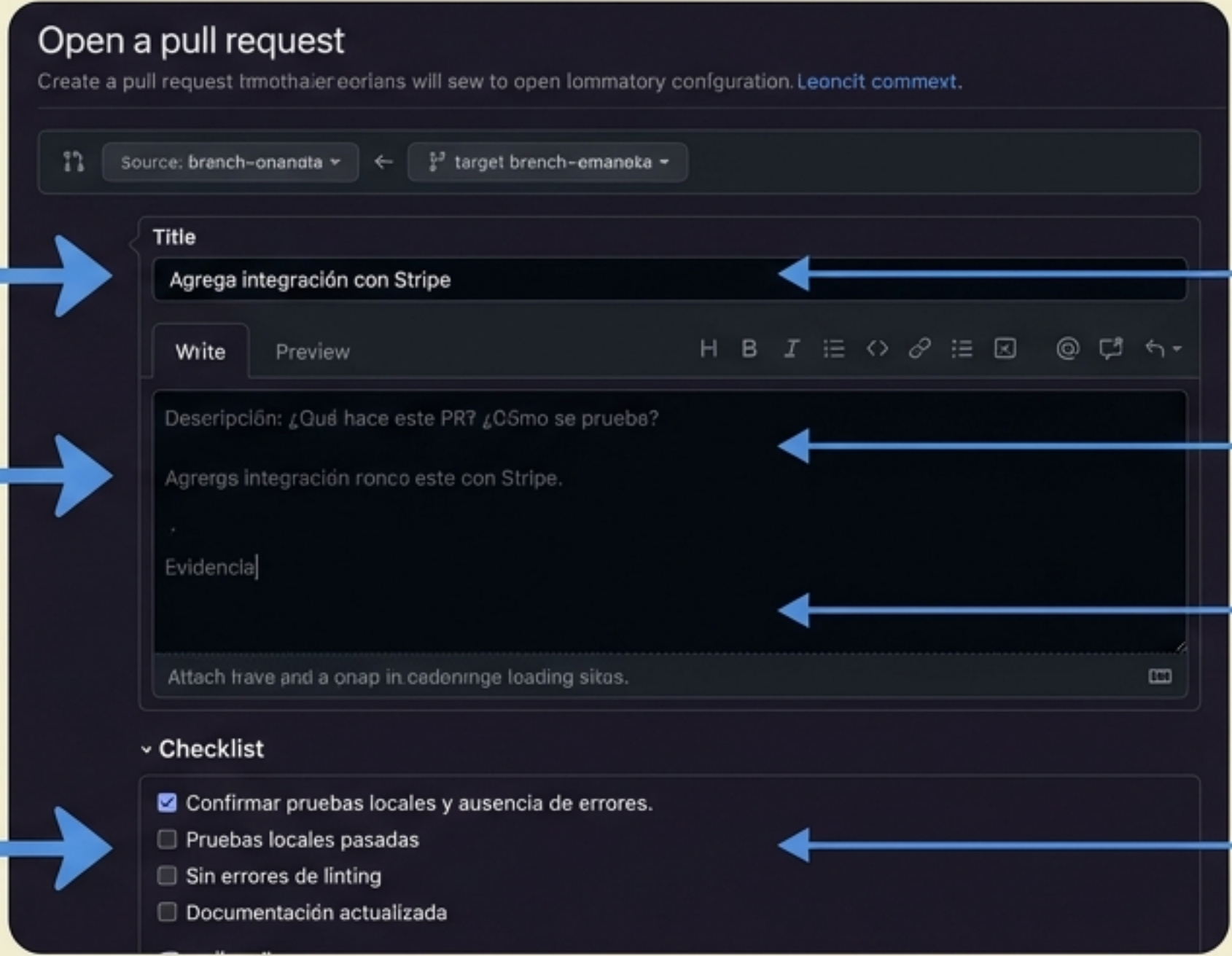
```
git checkout -b  
feature/nuevo-  
boton
```

```
git add +  
git commit
```

```
git push origin  
feature/nuevo-  
boton
```

Ir a GitHub y
crear el Pull
Request

Anatomía de un Pull Request Exitoso



The image shows a GitHub 'Open a pull request' form with several annotations pointing to specific fields:

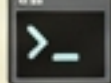
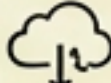
- Title:** 'Agrega integración con Stripe'. Annotation: **Título Claro:** "Agrega integración con Stripe".
- Descripción:** '¿Qué hace este PR? ¿Cómo se prueba?'. Annotation: **Descripción:** ¿Qué hace este PR? ¿Cómo se prueba?
- Evidencia:** 'Incluir capturas de pantalla o gifs.'. Annotation: **Evidencia:** Incluir capturas de pantalla o gifs.
- Checklist:** 'Confirmar pruebas locales y ausencia de errores.'. Annotation: **Checklist:** Confirmar pruebas locales y ausencia de errores.

The form also includes a 'Source' dropdown set to 'branch-onandta' and a 'target branch' dropdown set to 'branch-emaneka'. The 'Write' tab is selected, and the 'Preview' tab is visible. The 'Checklist' section is expanded, showing items like 'Confirmar pruebas locales y ausencia de errores.', 'Pruebas locales pasadas', 'Sin errores de linting', and 'Documentación actualizada'.

Meta: Facilitar la vida de quien va a revisar tu código.

Code Review y Refinamiento

El Proceso:

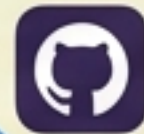
1.  **Feedback:** Tus compañeros comentarán mejoras necesarias.
2.  **Corrección:** Haces los cambios en tu computadora, haces commit y push.
3.  **Magia:** ✨ Los nuevos commits se agregan automáticamente al PR abierto. ¡No necesitas crear uno nuevo!

```
function procesarDatos(data) {  
  // Código complejo  
  for (let i = 0; i < data.length; i++) {  
    // Operaciones largas  
  }  
  return data;  
}  
  
function guardarEnBD(data) {  
  db.save(data);  
}
```

Función muy larga



Falta manejo de errores



Aprobación, Fusión y Limpieza



Fusión (Merge)

Squash and Merge: Combina todos tus pequeños commits en uno solo para mantener el historial limpio.



Limpieza

Una vez fusionado, la rama feature ya no es necesaria.

```
git checkout main  
git pull origin main # Baja la versión final  
git branch -d feature/nuevo-boton # Elimina rama local
```

Reglas de Oro en la Empresa



Commits Atómicos: Pequeños y frecuentes. Mejor 10 commits pequeños que 1 gigante.



Una Rama = Una Funcionalidad: No mezcles arreglos de bugs con nuevas funciones.



Nunca hagas Push a Main: Bloquea la rama principal; todo debe entrar vía Pull Request.



Sincroniza antes de empezar: Siempre ejecuta `'git pull'` al iniciar el día.

Consejos Finales para el Desarrollador Junior



- ✓ **No le temas a Git:** Es casi imposible romper algo que no se pueda arreglar.
- ↻ **Lee los errores:** Git es explícito; te dice qué está mal.
- ? **Pide ayuda:** Hasta los desarrolladores Senior googlean comandos.
- ⚙ **Práctica:** Crea un repositorio de prueba y rompe cosas. Es la única forma de aprender.

“Tu carrera profesional comienza con el primer ‘git init’.”